

Pointers in C (Part 1)

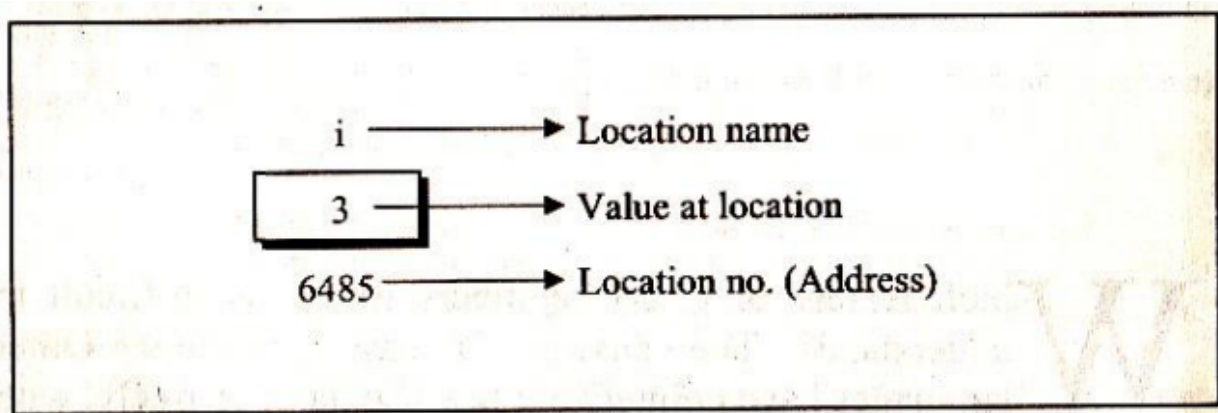
Consider the declamation,

```
int i=3;
```

This declaration tells the C Compiler to:

- (a) Reserve space in memory to hold the integer value.
- (b) Associate the name *i* with this memory location.
- (c) Store the value 3 at this location.

We may represent the *i*'s location in the memory by the following memory map:

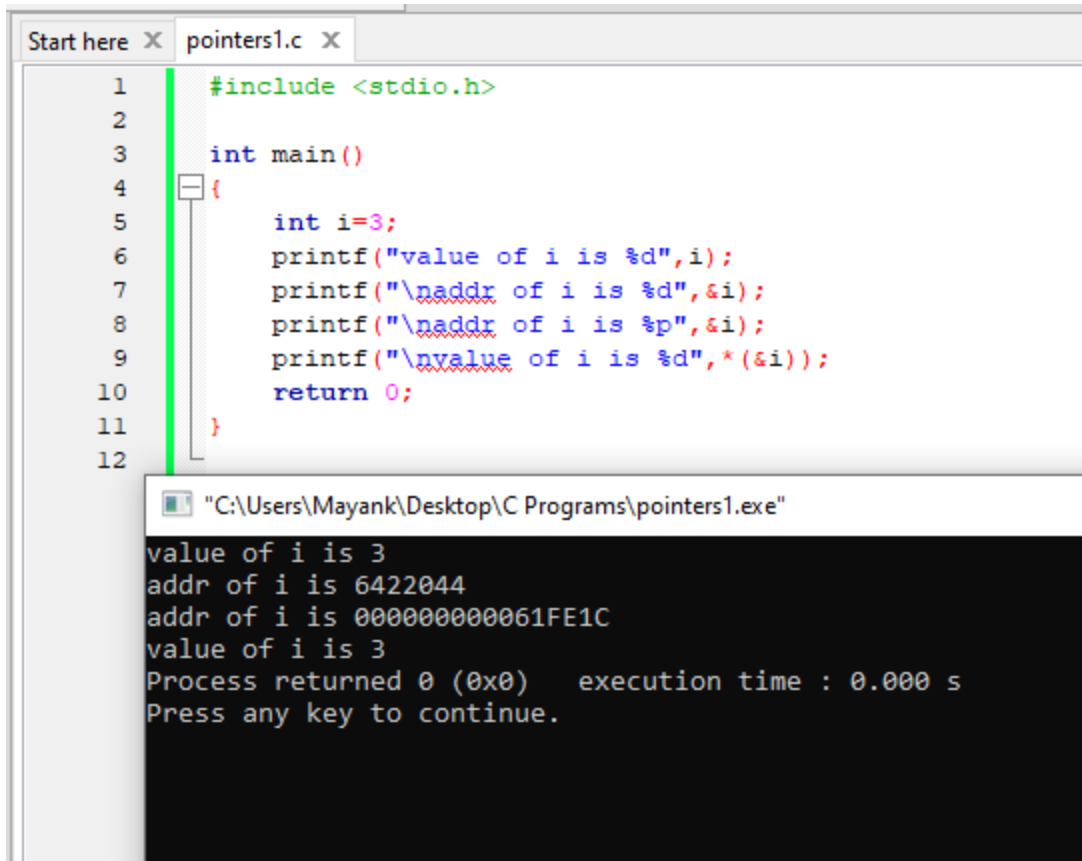


We see that the computer has selected memory location 6485 as the place to store the value 3. This location number 6485 is not a number to be relied upon, because some other time the computer may choose a different location for storing the value 3, the important point is, *i*'s address in memory is a number.

We can print this address through the program below:

```
#include <stdio.h>

int main()
{
    int i=3;
    printf("value of i is %d",i);
    printf("\naddr of i is %d",&i);
    printf("\naddr of i is %p",&i);
    printf("\nvalue of i is %d",*(&i));
    return 0;
}
```

The image shows a screenshot of a code editor window titled 'pointers1.c' and a separate console window titled '"C:\Users\Mayank\Desktop\C Programs\pointers1.exe"'. The code in the editor is a C program that includes <stdio.h>, defines a main function, declares an integer i with value 3, and prints its value, address, and the address of its address. The console output shows the program's execution, displaying the value of i as 3, its address as 6422044, and the address of its address as 0000000000061FE1C. The program returns 0 and the execution time is 0.000 s.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int i=3;
6      printf("value of i is %d",i);
7      printf("\naddr of i is %d",&i);
8      printf("\naddr of i is %p",&i);
9      printf("\nvalue of i is %d",*(&i));
10     return 0;
11 }
12
```

```
value of i is 3
addr of i is 6422044
addr of i is 0000000000061FE1C
value of i is 3
Process returned 0 (0x0)   execution time : 0.000 s
Press any key to continue.
```

look at the second printf() statement carefully. The & operator [the ' & ' operator used in this statement is C's 'address of' operator]. The expression &i returns the address of the variable i , which in this case happens to be 6422044. The pointer operator available in C is '*'. called 'value at address' operator. It returns the value stored at a particular address, 'value at address' operator is also called an 'indirection'.

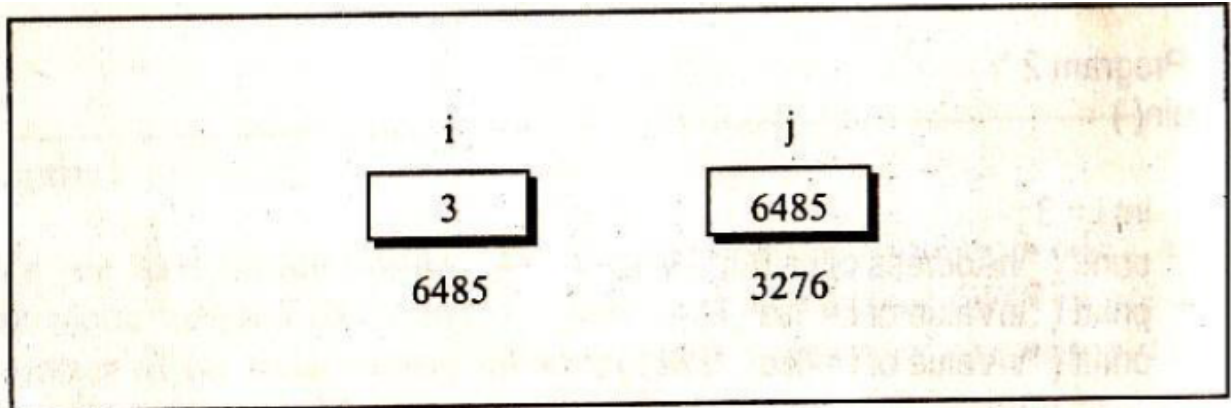
Let us now see what are pointers and how they can be used in various expressions. We have seen in the previous code that the expression &i returns the address of i. If we so desire, this address can be collected in a variable by saying,

```
j=&i;
```

But remember that J is not an ordinary variable like any other integer variable. It is a variable, which contains the address of another variable (i in this case)

Since j is a variable the compiler must provide it space in memory.

Once again, the following memory map would illustrate the contents of i and j.



```
#include <stdio.h>

int main()
{

int i= 3;
int *j ;
j=&i;
printf("\nvalue of i = %u", i );
printf("\nAddress of i = %u", &i );
printf( "\nAddress of j =%u", &j) ;
printf("\nvalue of j=%u",j);
printf( "\nvalue of i = %d",i);
printf( "\nvalue of i =%d",*(&i));
printf("\nvalue of i = %d",*j);
return 0;
}
```

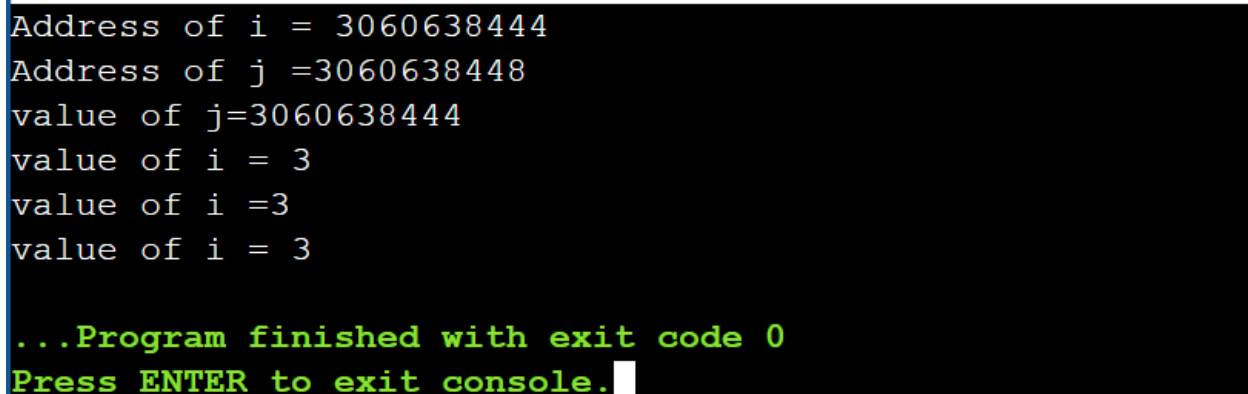
The output of the above program would be (considering the memory map above):

```
value of i=3
Address of i=6485
Address of j=3276
value of j = 6485
value of i = 3
value of i=3
value of i= 3
```

Go through the above program carefully taking help of the memory locations of i and j shown above. This program summarizes everything we have discussed so far. If you don't understand the program's output, or the meaning of the expressions &i, &j, *j and *(&i), reread the last few pages. Everything we say about pointers from here onwards will depend on your understanding of the expressions thoroughly.

Also see the output in online compiler:

```
1  #include <stdio.h>
2
3  int main()
4  {
5
6  int i= 3;
7  int *j ;
8  j=&i;
9  printf("\nvalue of i = %u", i );
10 printf("\nAddress of i = %u", &i );
11 printf( "\nAddress of j =%u", &j) ;
12 printf("\nvalue of j=%u",j);
13 printf( "\nvalue of i = %d",i);
14 printf( "\nvalue of i =%d",*(&i));
15 printf("\nvalue of i = %d",*j);
16 return 0;
17 }
18
```



```
Address of i = 3060638444
Address of j =3060638448
value of j=3060638444
value of i = 3
value of i =3
value of i = 3

...Program finished with exit code 0
Press ENTER to exit console.
```

We get a similar output.

Consider the code below:

```
#include <stdio.h>

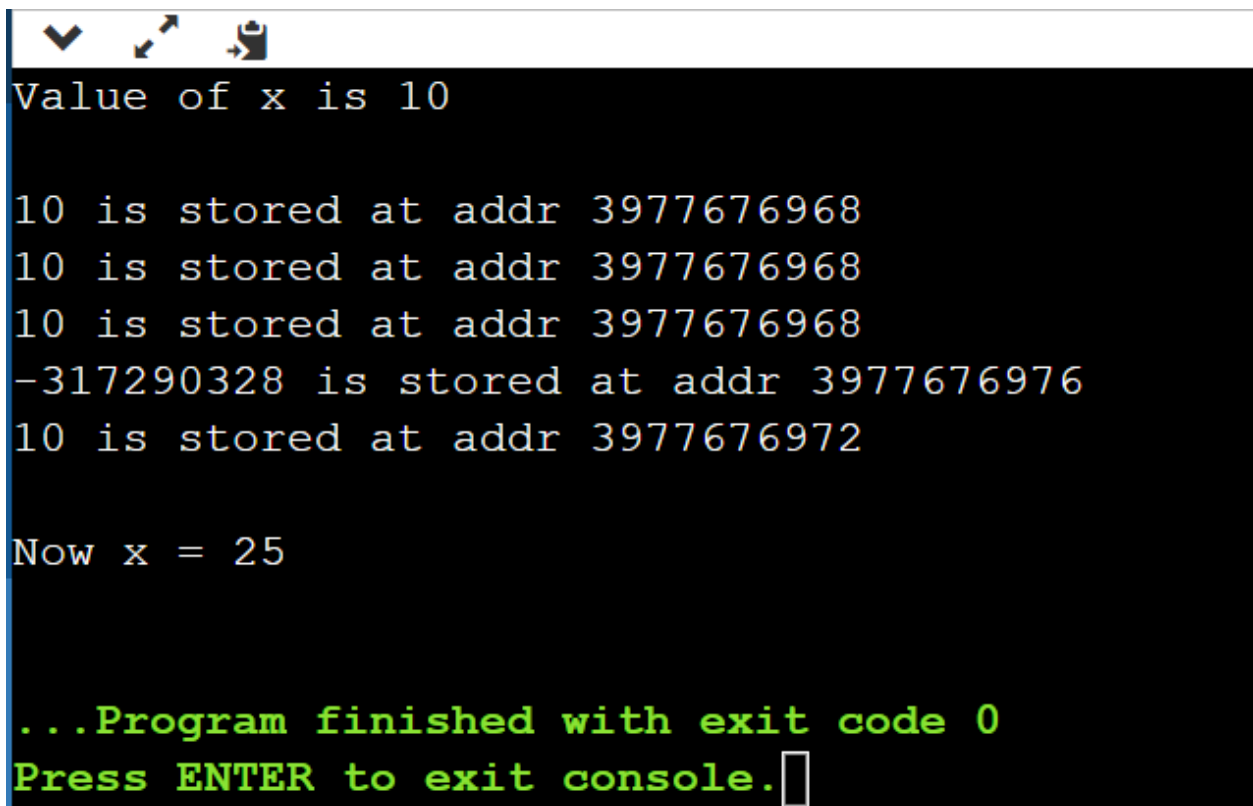
int main()
{
    int x, y;
    int *ptr;
    x = 10;
```

```

ptr = &x;
y = *ptr;
printf("Value of x is %d\n\n",x);
printf("%d is stored at addr %u\n", x, &x);
printf("%d is stored at addr %u\n", *&x, &x);
printf("%d is stored at addr %u\n", *ptr, ptr);
printf("%d is stored at addr %u\n", ptr, &ptr);
printf("%d is stored at addr %u\n", y, &y);
*ptr = 25;
printf("\nNow x = %d\n",x);

    return 0;
}

```



```

Value of x is 10

10 is stored at addr 3977676968
10 is stored at addr 3977676968
10 is stored at addr 3977676968
-317290328 is stored at addr 3977676976
10 is stored at addr 3977676972

Now x = 25

...Program finished with exit code 0
Press ENTER to exit console.

```

So, we can summarize this as follows:

- If you have a variable `var` in your program, `&var` will give you its address in the memory.
- We have used addresses numerous times while using the `scanf()` function:
`scanf("%d", &var);`
- **Pointers (pointer variables)** are special variables that are used to store addresses rather than values.
- **Pointer Syntax:** Here is how we can declare pointers.
`int* p;`
 Here, we have declared a pointer `p` of `int` type.
- You can also declare pointers in these ways.
`int *p1;`

```
int * p2;
```

- **Assigning addresses to Pointers**

Let's take an example.

```
int* pc, c;  
c = 5;  
pc = &c;
```

Here, 5 is assigned to the c variable. And, the address of c is assigned to the pc pointer.

- **Get Value of Thing Pointed by Pointers**

To get the value of the thing pointed by the pointers, we use the * operator. For example:

```
int* pc, c;  
c = 5;  
pc = &c;  
printf("%d", *pc);    // Output: 5
```

Here, the address of c is assigned to the pc pointer. To get the value stored in that address, we used *pc.

Note: In the above example, pc is a pointer, not *pc. You cannot and should not do something like *pc = &c;

- * is called the dereference operator (when working with pointers). It operates on a pointer and gives the value stored in that pointer.

- **Changing Value Pointed by Pointers**

Let's take an example.

```
int* pc, c;  
c = 5;  
pc = &c;  
c = 1;  
printf("%d", c);      // Output: 1  
printf("%d", *pc);    // Output: 1
```

We have assigned the address of c to the pc pointer.

Then, we changed the value of c to 1. Since pc and the address of c is the same, *pc gives us 1.

- **Example: Working of Pointers**

Let's take a working example.

```
#include <stdio.h>  
int main()  
{  
    int* pc, c;  
  
    c = 22;  
    printf("Address of c: %p\n", &c);
```

```

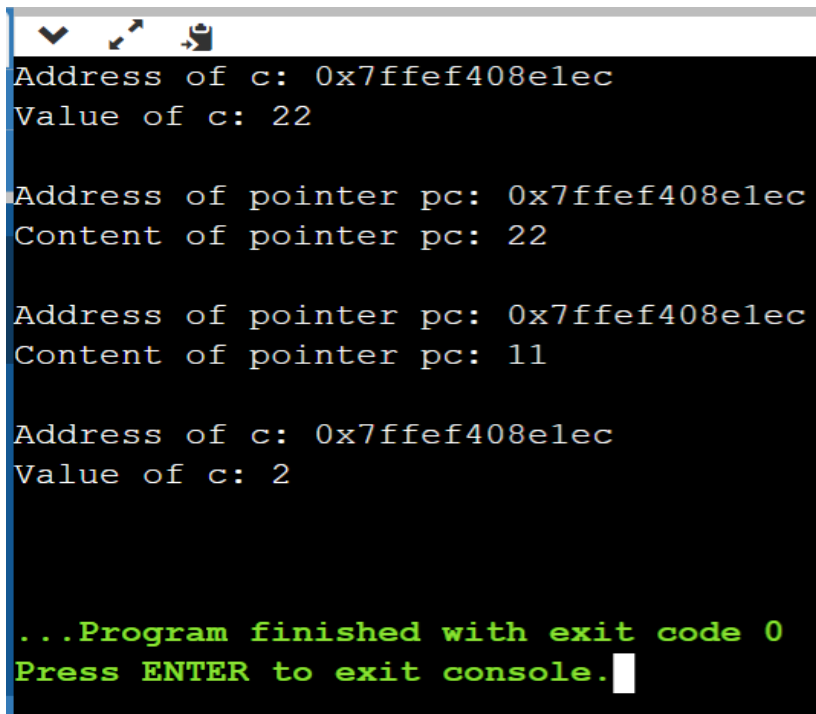
printf("Value of c: %d\n\n", c); // 22

pc = &c;
printf("Address of pointer pc: %p\n", pc);
printf("Content of pointer pc: %d\n\n", *pc); // 22

c = 11;
printf("Address of pointer pc: %p\n", pc);
printf("Content of pointer pc: %d\n\n", *pc); // 11

*pc = 2;
printf("Address of c: %p\n", &c);
printf("Value of c: %d\n\n", c); // 2
return 0;
}

```



```

Address of c: 0x7ffef408elec
Value of c: 22

Address of pointer pc: 0x7ffef408elec
Content of pointer pc: 22

Address of pointer pc: 0x7ffef408elec
Content of pointer pc: 11

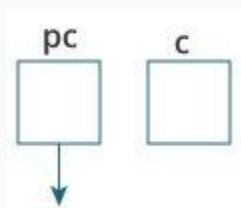
Address of c: 0x7ffef408elec
Value of c: 2

...Program finished with exit code 0
Press ENTER to exit console.

```

Explanation of the program

1. `int* pc, c;`

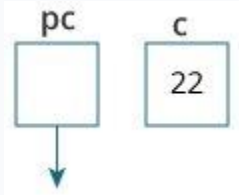


Here, a pointer `pc` and a normal variable `c`, both of type `int`, is created.

Since `pc` and `c` are not initialized initially, pointer `pc` points to either no address or a

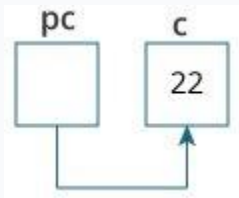
random address. And, variable `c` has an address but contains random garbage value.

2. `c = 22;`



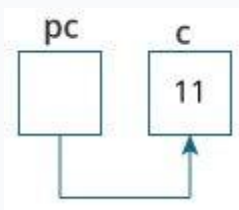
This assigns 22 to the variable `c`. That is, 22 is stored in the memory location of variable `c`.

3. `pc = &c;`



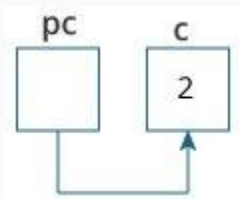
This assigns the address of variable `c` to the pointer `pc`.

4. `c = 11;`



This assigns 11 to variable `c`.

5. `*pc = 2;`



This changes the value at the memory location pointed by the pointer `pc` to 2.

Common mistakes when working with pointers

Suppose, you want a pointer `pc` to point to the address of `c`. Then,

```
int c, *pc;

// pc is address but c is not
pc = c; // Error

// &c is address but *pc is not
*pc = &c; // Error

// both &c and pc are addresses
pc = &c; // Not an error

// both c and *pc are values
*pc = c; // Not an error
```

Here's an example of pointer syntax beginners often find confusing.

```
#include <stdio.h>
int main() {
    int c = 5;
    int *p = &c;

    printf("%d", *p); // 5
    return 0;
}
```

Why didn't we get an error when using `int *p = &c;`?

It's because

```
int *p = &c;
```

is equivalent to

```
int *p;
p = &c;
```

In both cases, we are creating a pointer `p` (not `*p`) and assigning `&c` to it.

To avoid this confusion, we can use the statement like this:

```
int* p = &c;
```

Pointer Arithmetic

We can perform arithmetic operations on the pointers like addition, subtraction, etc. However, as we know that a pointer contains the address, the result of an arithmetic operation performed on

the pointer will also be a pointer if the other operand is of type integer. In pointer-from-pointer subtraction, the result will be an integer value. Following arithmetic operations are possible on the pointer in C language:

- Increment
- Decrement
- Addition
- Subtraction
- Comparison

Incrementing Pointer in C

If we increment a pointer by 1, the pointer will start pointing to the immediate next location. This is somewhat different from the general arithmetic since the value of the pointer will get increased by the size of the data type to which the pointer is pointing.

We can traverse an array by using the increment operation on a pointer which will keep pointing to every element of the array, perform some operation on that, and update itself in a loop.

The Rule to increment the pointer is given below:

`new_address= current_address + i * size_of(data type)`

Where i is the number by which the pointer get increased.

32-bit

For a 32-bit int variable, it will be incremented by 2 bytes.

64-bit

For a 64-bit int variable, it will be incremented by 4 bytes.

Let's see the example of incrementing a pointer variable on 64-bit architecture.

```
#include <stdio.h>

int main()
{

    int number=50;
    int *p;//pointer to int
    p=&number;//stores the address of number variable
    printf("Address of p variable is %u \n",p);
    p=p+1;
    printf("After increment: Address of p variable is %u \n",p); // in our
    case, p will get incremented by 4 bytes.

    return 0;
}
```

```
1 #include <stdio.h>
2
3 int main()
4 {
5
6 int number=50;
7 int *p;//pointer to int
8 p=&number;//stores the address of number variable
9 printf("Address of p variable is %u \n",p);
10 p=p+1;
11 printf("After increment: Address of p variable is %u \n",p);
12
13 return 0;
14 }
15
16
```

input

Address of p variable is 3347981868
After increment: Address of p variable is 3347981872
...Program finished with exit code 0
Press ENTER to exit console.

Decrementing Pointer in C

Like increment, we can decrement a pointer variable. If we decrement a pointer, it will start pointing to the previous location. The formula of decrementing the pointer is given below:

$$\text{new_address} = \text{current_address} - i * \text{size_of}(\text{data type})$$

32-bit

For a 32-bit int variable, it will be decremented by 2 bytes.

64-bit

For a 64-bit int variable, it will be decremented by 4 bytes.

Let's see an example of decrementing a pointer variable on 64-bit OS.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int number=50;
```

```

int *p;//pointer to int
p=&number;//stores the address of number variable
printf("Address of p variable is %u \n",p);
p=p-1;
printf("After decrement: Address of p variable is %u \n",p);
// P will now point to the immediate previous location.
return 0;
}

```

```

1  #include <stdio.h>
2
3  int main()
4  {
5  int number=50;
6  int *p;//pointer to int
7  p=&number;//stores the address of number variable
8  printf("Address of p variable is %u \n",p);
9  p=p-1;
10 printf("After decrement: Address of p variable is %u \n",p);
11 // P will now point to the immediate previous location.
12     return 0;
13 }
14
15

```

input

```

|
|
|
|
Address of p variable is 3298699996
After decrement: Address of p variable is 3298699992

...Program finished with exit code 0
Press ENTER to exit console.

```

Pointer Addition: adding a numeric value to a pointer variable

We can add a value to the pointer variable. The formula of adding value to pointer is given below:

`new_address= current_address + (number * size_of(data type))`

32-bit

For 32-bit int variable, it will add 2 * number.

64-bit

For 64-bit int variable, it will add 4 * number.

Let's see the example of adding value to a pointer variable on 64-bit architecture.

```
#include <stdio.h>
```

```
int main()
{
    int number=50;
    int *p;//pointer to int
    p=&number;//stores the address of number variable
    printf("Address of p variable is %u \n",p);
    p=p+3;    //adding 3 to pointer variable
    printf("After adding 3: Address of p variable is %u \n",p);

    return 0;
}
```

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int number=50;
6      int *p;//pointer to int
7      p=&number;//stores the address of number variable
8      printf("Address of p variable is %u \n",p);
9      p=p+3;    //adding 3 to pointer variable
10     printf("After adding 3: Address of p variable is %u \n",p);
11
12     return 0;
13 }
14
15
```

input

```
Address of p variable is 252531884
After adding 3: Address of p variable is 252531896

...Program finished with exit code 0
Press ENTER to exit console.
```

Pointer Subtraction: subtracting a numeric value to a pointer variable

Like pointer addition, we can subtract a value from the pointer variable. Subtracting any number from a pointer will give an address. The formula of subtracting value from the pointer variable is given below:

$$\text{new_address} = \text{current_address} - (\text{number} * \text{size_of}(\text{data type}))$$

32-bit

For 32-bit int variable, it will subtract $2 * \text{number}$.

64-bit

For 64-bit int variable, it will subtract $4 * \text{number}$.

Let's see the example of subtracting value from the pointer variable on 64-bit architecture:

```
#include <stdio.h>
```

```
int main()
{
    int number=50;
    int *p;//pointer to int
    p=&number;//stores the address of number variable
    printf("Address of p variable is %u \n",p);
    p=p-3; //subtracting 3 from pointer variable
    printf("After subtracting 3: Address of p variable is %u \n",p);

    return 0;
}
```

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int number=50;
6      int *p;//pointer to int
7      p=&number;//stores the address of number variable
8      printf("Address of p variable is %u \n",p);
9      p=p-3; //subtracting 3 from pointer variable
10     printf("After subtracting 3: Address of p variable is %u \n",p);
11
12     return 0;
13 }
14
15

```

```

input
Address of p variable is 1213342396
After subtracting 3: Address of p variable is 1213342384
...Program finished with exit code 0
Press ENTER to exit console.

```

However, instead of subtracting a number, **we can also subtract an address from another address (pointer). This will result in a number.** It will not be a simple arithmetic operation, but it will follow the following rule.

If two pointers are of the same type,

$\text{Address2} - \text{Address1} = (\text{Subtraction of two addresses}) / \text{size of data type which pointer points}$

Consider the following example to subtract one pointer from another.

```

#include<stdio.h>
void main ()
{
    int i = 100;
    int *p = &i;
    int *temp;
    temp = p;
    p = p + 3;
    printf("Pointer Subtraction: %u - %u = %d",p, temp, p-temp);
}

```

```

1  #include<stdio.h>
2  void main ()
3  {
4      int i = 100;
5      int *p = &i;
6      int *temp;
7      temp = p;
8      p = p + 3;
9      printf("Pointer Subtraction: %u - %u = %d",p, temp, p-temp);
10 }
11 |

```

input

```

Pointer Subtraction: 3898092272 - 3898092260 = 3
...Program finished with exit code 0
Press ENTER to exit console.

```

Illegal arithmetic with pointers

There are various operations which can not be performed on pointers. Since, pointers store addresses hence we must ignore the operations which may lead to an illegal address, for example, addition, and multiplication. A list of such operations is given below.

- Address + Address = illegal
- Address * Address = illegal
- Address % Address = illegal
- Address / Address = illegal
- Address & Address = illegal
- Address ^ Address = illegal
- Address | Address = illegal
- ~Address = illegal

Relationship Between Arrays and Pointers

An array is a block of sequential data. Let's write a program to print addresses of array elements.

```
#include <stdio.h>
```

```
int main() {  
    int x[4];  
    int i;  
  
    for(i = 0; i < 4; ++i) {  
        printf("&x[%d] = %p\n", i, &x[i]);  
    }  
  
    printf("Address of array x: %p", x);  
  
    return 0;  
}
```

Output

```

1  #include <stdio.h>
2  int main() {
3      int x[4];
4      int i;
5
6      for(i = 0; i < 4; ++i) {
7          printf("&x[%d] = %p\n", i, &x[i]);
8      }
9
10     printf("Address of array x: %p", x);
11
12     return 0;
13 }

```

```

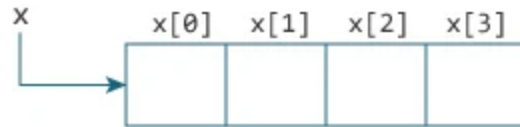
&x[0] = 0x7ffcedec79d0
&x[1] = 0x7ffcedec79d4
&x[2] = 0x7ffcedec79d8
&x[3] = 0x7ffcedec79dc
Address of array x: 0x7ffcedec79d0

...Program finished with exit code 0
Press ENTER to exit console.

```

There is a difference of 4 bytes between two consecutive elements of array x. It is because the size of int is 4 bytes (on our compiler).

Notice that the address of &x[0] and x is the same. It's because the variable name x points to the first element of the array.



Relation between Arrays and Pointers

From the above example, it is clear that `&x[0]` is equivalent to `x`. And, `x[0]` is equivalent to `*x`.

Similarly,

`&x[1]` is equivalent to `x+1` and `x[1]` is equivalent to `*(x+1)`.

`&x[2]` is equivalent to `x+2` and `x[2]` is equivalent to `*(x+2)`.

...

Basically, `&x[i]` is equivalent to `x+i` and `x[i]` is equivalent to `*(x+i)`.

Here are some examples to understand better:

```
#include <stdio.h>
int main() {

    int i, x[6], sum = 0;

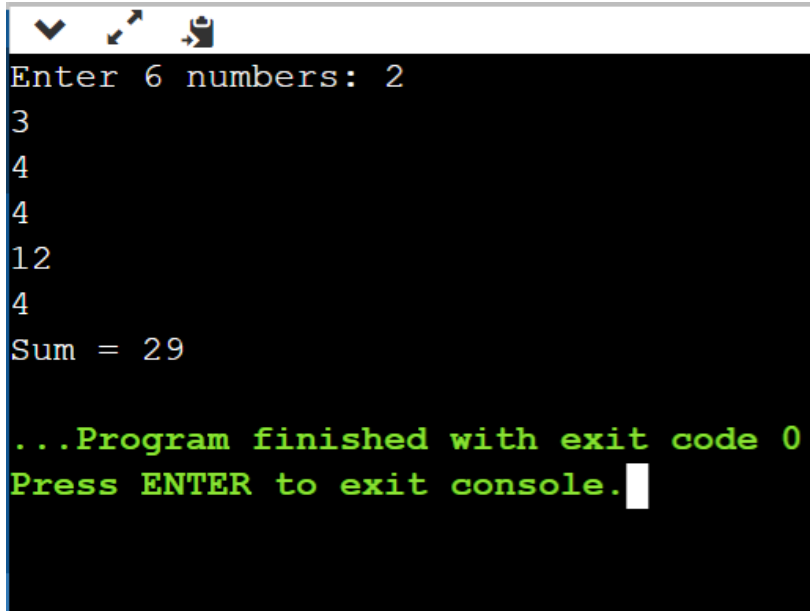
    printf("Enter 6 numbers: ");

    for(i = 0; i < 6; ++i) {
        // Equivalent to scanf("%d", &x[i]);
        scanf("%d", x+i);

        // Equivalent to sum += x[i]
        sum += *(x+i);
    }

    printf("Sum = %d", sum);

    return 0;
}
```



```
Enter 6 numbers: 2
3
4
4
12
4
Sum = 29

...Program finished with exit code 0
Press ENTER to exit console.
```

Here, we have declared an array x of 6 elements. To access elements of the array, we have used pointers.

Another Example:

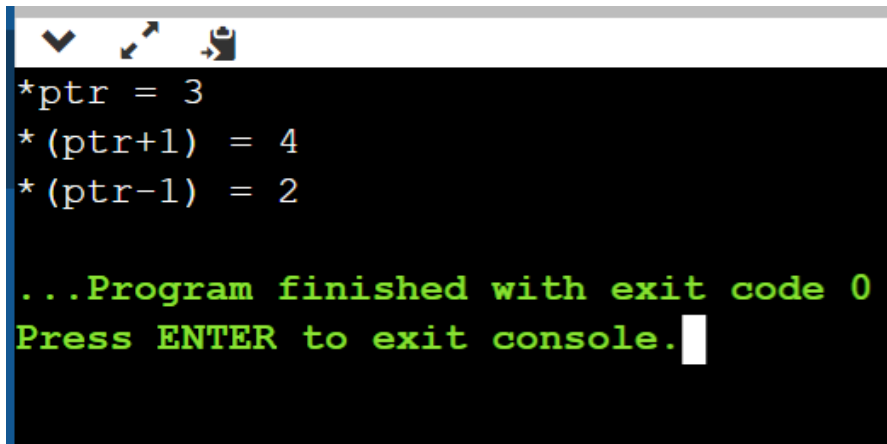
```
#include <stdio.h>
int main() {

    int x[5] = {1, 2, 3, 4, 5};
    int* ptr;

    // ptr is assigned the address of the third element
    ptr = &x[2];

    printf("*ptr = %d \n", *ptr);    // 3
    printf("* (ptr+1) = %d \n", *(ptr+1)); // 4
    printf("* (ptr-1) = %d", *(ptr-1)); // 2

    return 0;
}
```

A terminal window with a dark background. At the top, there are three icons: a checkmark, a cursor, and a document. The text in the terminal is as follows:

```
*ptr = 3
*(ptr+1) = 4
*(ptr-1) = 2

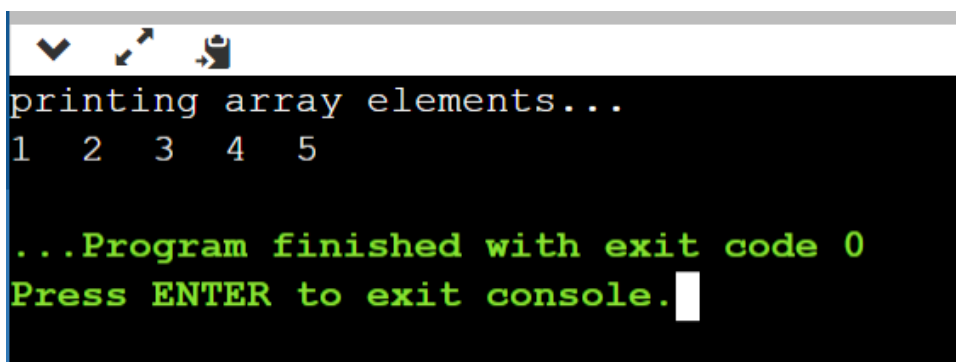
...Program finished with exit code 0
Press ENTER to exit console.
```

In this example, `&x[2]`, the address of the third element, is assigned to the `ptr` pointer. Hence, 3 was displayed when we printed `*ptr`.

And, printing `*(ptr+1)` gives us the fourth element. Similarly, printing `*(ptr-1)` gives us the second element.

Another Example:

```
#include<stdio.h>
void main ()
{
    int arr[5] = {1, 2, 3, 4, 5};
    int *p = arr;
    int i;
    printf("printing array elements...\n");
    for(i = 0; i< 5; i++)
    {
        printf("%d  ",*(p+i));
    }
}
```

A terminal window with a dark background. At the top, there are three icons: a checkmark, a cursor, and a document. The text in the terminal is as follows:

```
printing array elements...
1  2  3  4  5

...Program finished with exit code 0
Press ENTER to exit console.
```

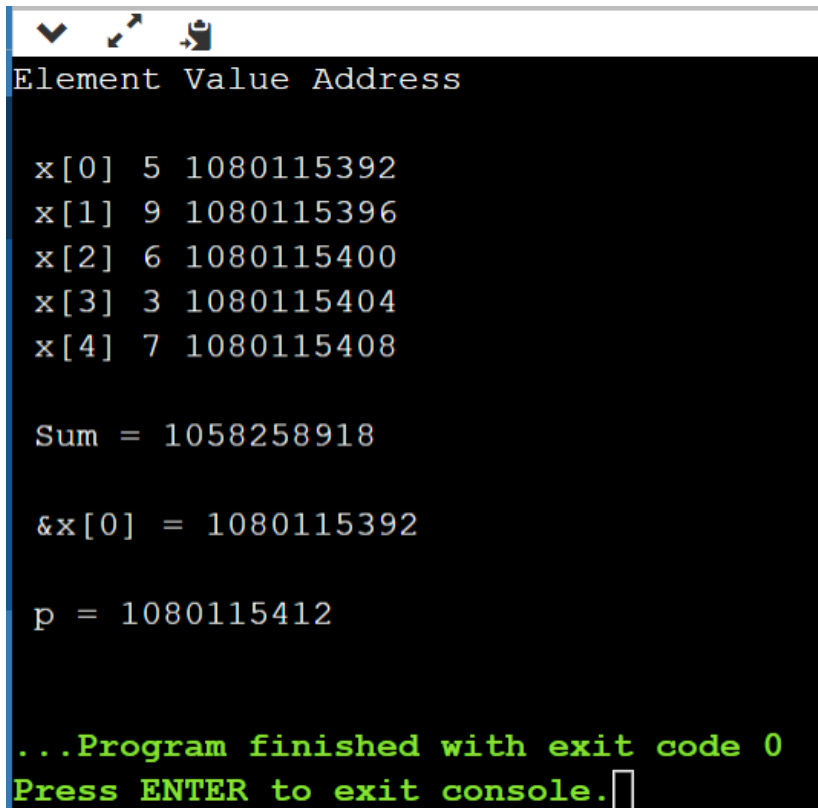
Another Example:

```
#include <stdio.h>
```

```

int main()
{
    int *p, sum, i;
    int x[5] = {5,9,6,3,7};
    i = 0;
    p = x; /* initializing with base address of x */
    printf("Element Value Address\n\n");
    while(i < 5)
    {
        printf(" x[%d] %d %u\n", i, *p, p);
        sum = sum + *p; /* accessing array element */
        i++, p++; /* incrementing pointer */
    }
    printf("\n Sum = %d\n", sum);
    printf("\n &x[0] = %u\n", &x[0]);
    printf("\n p = %u\n", p);
    return 0;
}

```



```

Element Value Address

x[0] 5 1080115392
x[1] 9 1080115396
x[2] 6 1080115400
x[3] 3 1080115404
x[4] 7 1080115408

Sum = 1058258918

&x[0] = 1080115392

p = 1080115412

...Program finished with exit code 0
Press ENTER to exit console.

```

Now as the last example we create three programs to print elements of an array and their addresses, basically all give similar output but use different expressions in printf() to print the same things:

Code 1:

```

#include <stdio.h>
int main( )
{

```

```

int val[7] = { 11, 22, 33, 44, 55, 66, 77 } ;
/* for loop to print value and address of each element of array*/
for ( int i = 0 ; i < 7 ; i++ )
{
    printf("val[%d]: value is %d and address is %u\n", i, val[i],
&val[i]);
}
return 0;
}

```

```

val[0]: value is 11 and address is 1262926320
val[1]: value is 22 and address is 1262926324
val[2]: value is 33 and address is 1262926328
val[3]: value is 44 and address is 1262926332
val[4]: value is 55 and address is 1262926336
val[5]: value is 66 and address is 1262926340
val[6]: value is 77 and address is 1262926344

...Program finished with exit code 0
Press ENTER to exit console.

```

Code 2:

```

#include <stdio.h>
int main( )
{
    /*Pointer variable*/
    int *p;

    /*Array declaration*/
    int val[7] = { 11, 22, 33, 44, 55, 66, 77 } ;

    /* Assigning the address of val[0] the pointer
    * You can also write like this:
    * p = var;
    * because array name represents the address of the first element
    */
    p = &val[0];

    for ( int i = 0 ; i<7 ; i++ )
    {
        printf("val[%d]: value is %d and address is %u\n", i, *p, p);
    }
}

```

```

        /* Incrementing the pointer so that it points to next element
        * on every increment.
        */
        p++;
    }
    return 0;
}

```

```

val[0]: value is 11 and address is 2054505120
val[1]: value is 22 and address is 2054505124
val[2]: value is 33 and address is 2054505128
val[3]: value is 44 and address is 2054505132
val[4]: value is 55 and address is 2054505136
val[5]: value is 66 and address is 2054505140
val[6]: value is 77 and address is 2054505144

...Program finished with exit code 0
Press ENTER to exit console.

```

Code 3:

```

#include <stdio.h>
int main( )
{
    int *p;
    int val[7] = { 11, 22, 33, 44, 55, 66, 77 } ;
    p = val;
    for ( int i = 0 ; i<7 ; i++ )
    {
        printf("val[%d]: value is %d and address is %p\n", i, *(p+i), (p+i));
    }
    return 0;
}

```



```
val[0]: value is 11 and address is 0x7ffc16b95340
val[1]: value is 22 and address is 0x7ffc16b95344
val[2]: value is 33 and address is 0x7ffc16b95348
val[3]: value is 44 and address is 0x7ffc16b9534c
val[4]: value is 55 and address is 0x7ffc16b95350
val[5]: value is 66 and address is 0x7ffc16b95354
val[6]: value is 77 and address is 0x7ffc16b95358
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```